

AI ASSISTED CODING

LAB-2

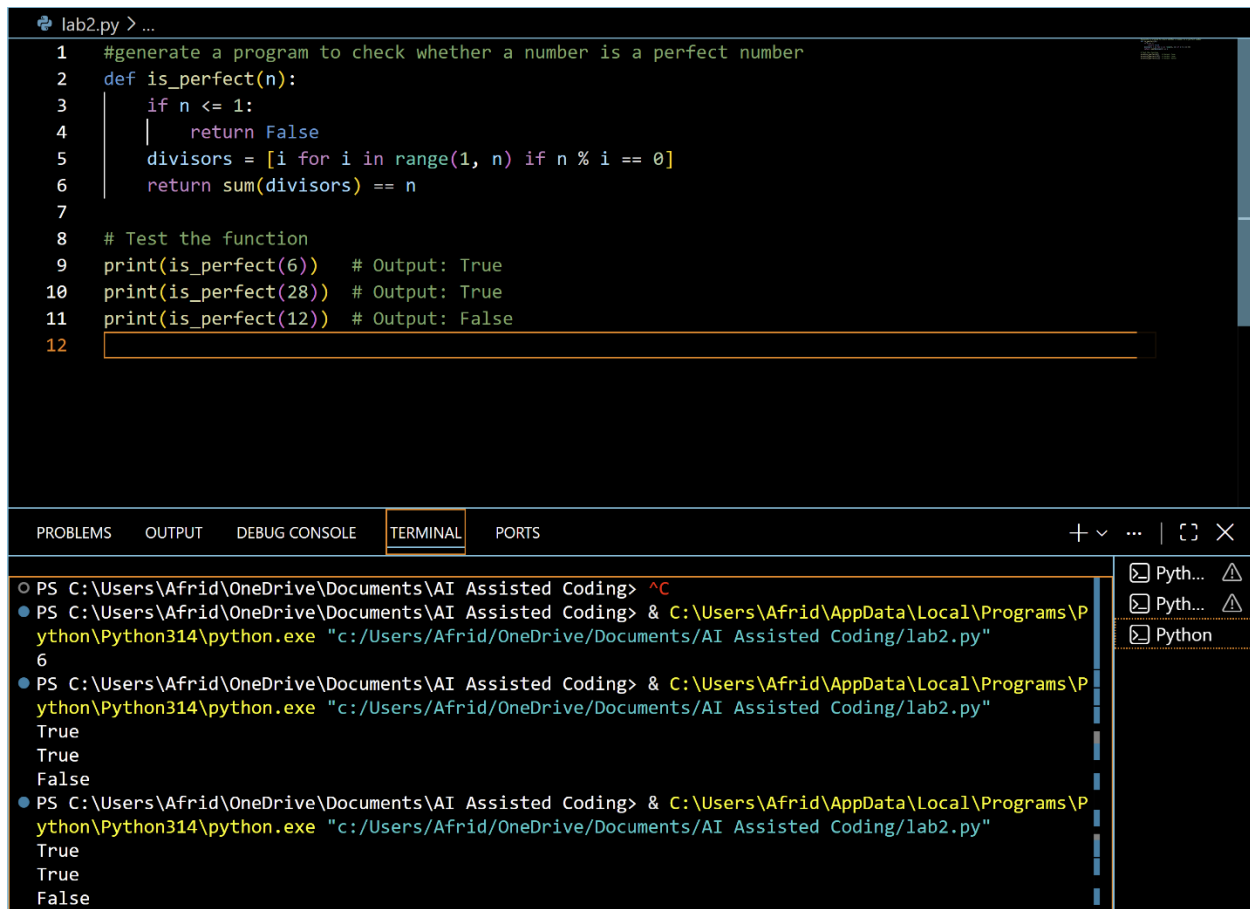
NAME: Shaik Afrid

ROLL NO : 2503A52385

BATCH: 13

1. AI TOOL USED : VS CODE

PROMT: #generate a code to check whether a number is perfect numbers



The screenshot shows the Visual Studio Code interface. The editor window displays a Python file named 'lab2.py' with the following code:

```
1 #generate a program to check whether a number is a perfect number
2 def is_perfect(n):
3     if n <= 1:
4         return False
5     divisors = [i for i in range(1, n) if n % i == 0]
6     return sum(divisors) == n
7
8 # Test the function
9 print(is_perfect(6)) # Output: True
10 print(is_perfect(28)) # Output: True
11 print(is_perfect(12)) # Output: False
12
```

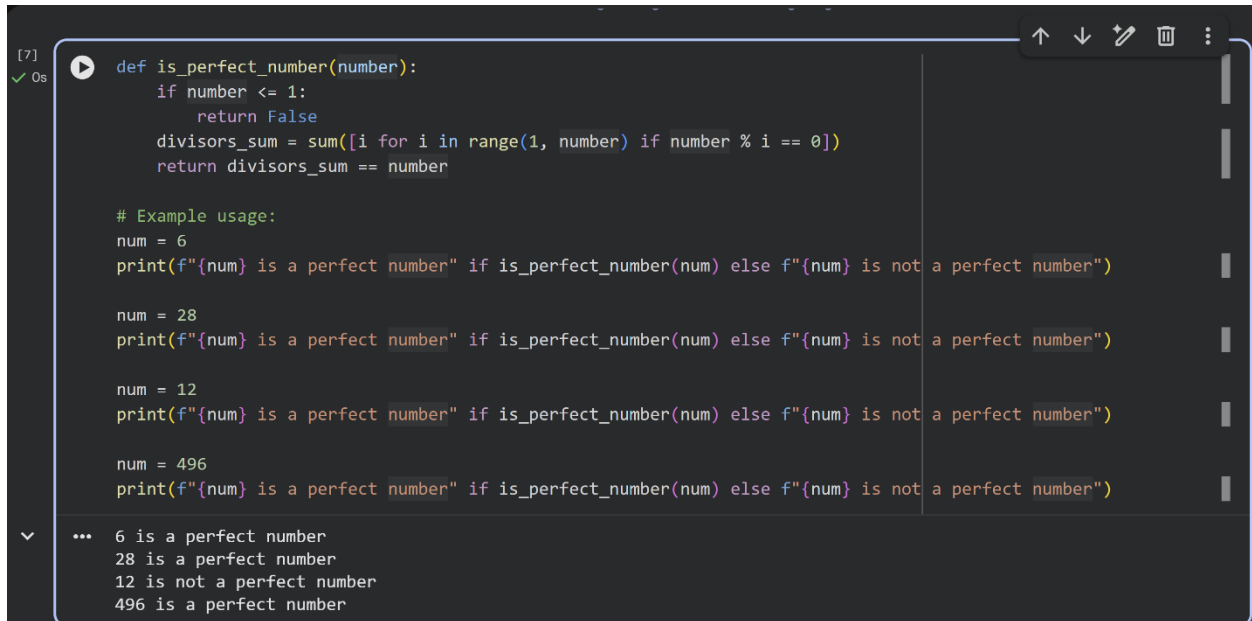
The bottom panel shows the 'TERMINAL' tab with the following commands and output:

```
PS C:\Users\Afrid\OneDrive\Documents\AI Assisted Coding> ^C
PS C:\Users\Afrid\OneDrive\Documents\AI Assisted Coding> & C:\Users\Afrid\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/Afrid/OneDrive/Documents/AI Assisted Coding/lab2.py"
6
PS C:\Users\Afrid\OneDrive\Documents\AI Assisted Coding> & C:\Users\Afrid\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/Afrid/OneDrive/Documents/AI Assisted Coding/lab2.py"
True
True
False
PS C:\Users\Afrid\OneDrive\Documents\AI Assisted Coding> & C:\Users\Afrid\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/Afrid/OneDrive/Documents/AI Assisted Coding/lab2.py"
True
True
False
```

On the right side of the terminal, there are three Python-related icons: 'Pyth...', 'Pyth...', and 'Python'.

2.AI TOOL USED : GOOGLE COLAB

PROMT: #generate a code to check whether a number is perfect numbers



```
[7] ✓ 0s
def is_perfect_number(number):
    if number <= 1:
        return False
    divisors_sum = sum([i for i in range(1, number) if number % i == 0])
    return divisors_sum == number

# Example usage:
num = 6
print(f"{num} is a perfect number" if is_perfect_number(num) else f"{num} is not a perfect number")

num = 28
print(f"{num} is a perfect number" if is_perfect_number(num) else f"{num} is not a perfect number")

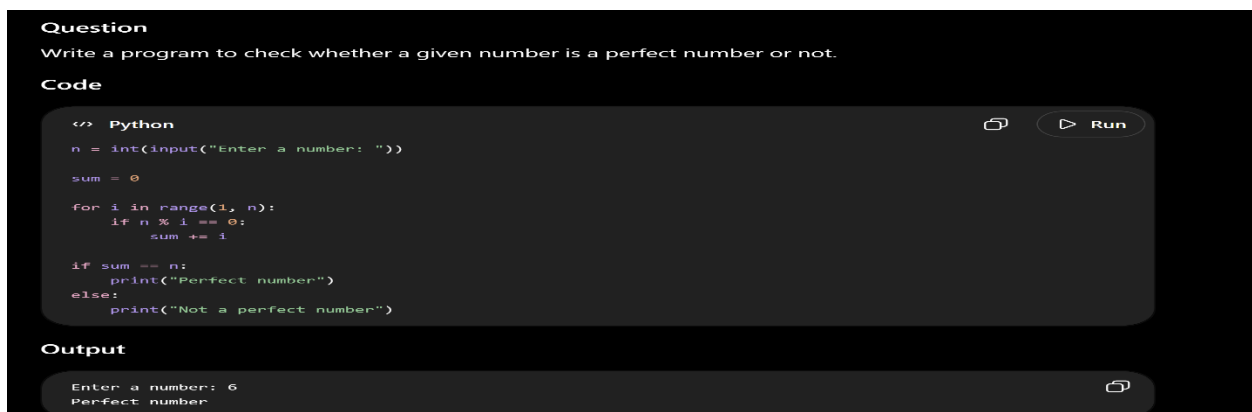
num = 12
print(f"{num} is a perfect number" if is_perfect_number(num) else f"{num} is not a perfect number")

num = 496
print(f"{num} is a perfect number" if is_perfect_number(num) else f"{num} is not a perfect number")

... 6 is a perfect number
    28 is a perfect number
    12 is not a perfect number
    496 is a perfect number
```

3.AI TOOL USED : CHATGPT

PROMT#generate a code to check whether a number is perfect numbers



Question
Write a program to check whether a given number is a perfect number or not.

Code

```
Python
n = int(input("Enter a number: "))
sum = 0
for i in range(1, n):
    if n % i == 0:
        sum += i
if sum == n:
    print("Perfect number")
else:
    print("Not a perfect number")
```

Output

```
Enter a number: 6
Perfect number
```

Comparison

The **1st code** uses a function `is_perfect(n)` and a list comprehension to find all the divisors of a number. It is reusable and gives True or False as the result.

The **2nd code** also uses a function, but it demonstrates the function with multiple numbers such as **6, 28, 12, and 496**. It gives a clear message like "6 is a perfect number," so it is better for demonstrating different test cases.

The **3rd code** does not use a function. It directly takes a number from the user, checks its divisors using a simple for loop, and prints whether it is a perfect number or not. This makes it easier for beginners to understand.

CONCLUSION

The **3rd code is the easiest and most beginner-friendly** because it uses simple concepts like `input()`, for loop, if, and sum.

The **1st code is better for understanding functions and list comprehensions**, while the **2nd code is useful for demonstrating a reusable function with multiple examples**.

So, for a **beginner-level assignment**, the **3rd code is the simplest choice**.